

On the Combinatorial Complexity of Bounded Sparse Learning with Errors (sLWE)

Abul Kalam and Santanu Sarkar and Willi Meier

the date of receipt and acceptance should be inserted later

Abstract Sparse Learning With Errors (sLWE), introduced at CRYPTO 2024 by Jain et al., is a recent hardness assumption aimed at improving the efficiency–security trade-off of lattice-based cryptography in the post-quantum setting. In their paper, the authors posed the open problem of understanding the security of sparse instances with small secrets. Motivated by this question, we study a bounded variant of sLWE in which both the secret and error vectors are restricted to the interval $\{-B, \dots, B\}$. We develop a combinatorial cryptanalytic framework based on subsystem extraction combined with a meet-in-the-middle strategy, enabling recovery of bounded secret vectors. Our analysis provides a detailed complexity evaluation across different sparsity and modulus regimes and yields concrete security estimates for the bounded setting.

Our work does not claim any attack on the proposal of Jain et al.; instead, it provides a concrete analysis of sparse LWE with bounded secrets and errors, together with explicit complexity bounds showing reduced security margins in this setting. Our results indicate that security levels comparable to dimension-1024 LWE for the proposed parameter sets do not necessarily carry over to bounded sLWE, with the ternary case ($B = 1$) providing a representative example. In this ternary setting, for modulus 2^{64} the secret can be recovered within practical complexity bounds, while for moduli 2^{32} and 2^{16} we observe a substantial reduction in the effective security margin. These findings provide a systematic cryptanalytic study of bounded sLWE and clarify how sparsity influences the security of LWE-type constructions.

Keywords Sparse Learning With Errors (sLWE) · Bounded sLWE · Combinatorial Attack · Post-Quantum Cryptography.

Mathematics Subject Classification (2000) 94A60

A. Kalam · S. Sarkar
Department of Mathematics,
Indian Institute of Technology Madras,
Chennai, Tamil Nadu 600036, India.
E-mail: abulkalam.sunny@gmail.com, sarkar.santanu.bir1@gmail.com

W. Meier
University of Applied Sciences and Arts Northwestern Switzerland (FHNW),
Windisch, Switzerland.
E-mail: willimeier48@gmail.com

1 Introduction

In recent years, the study of lattice-based cryptography has become increasingly important, particularly in light of the growing concerns about quantum computing’s potential to undermine traditional security methods. Lattice-based schemes, such as those based on the Learning With Errors (LWE) problem, are viewed as promising candidates for post-quantum cryptography due to their mathematical foundations that appear resistant to quantum attacks. Introduced by Regev in 2005 [19], the LWE problem serves as a cornerstone in this field, providing a robust framework for constructing secure cryptographic primitives. An LWE instance, represented as $(\mathbf{A}, \mathbf{b}) \in \mathbb{Z}_M^{m \times n} \times \mathbb{Z}_M^m$, involves a matrix $\mathbf{A}$ generated randomly over $\mathbb{Z}_M$. The primary challenge lies in recovering a hidden secret vector $\mathbf{s} \in \mathbb{Z}_M^n$ from a noisy equation $\mathbf{A}\mathbf{s} = \mathbf{b} + \mathbf{e} \pmod{M}$, where $\mathbf{e}$ represents a carefully introduced error term. The difficulty of this problem underpins many cryptographic tools, such as encryption schemes, digital signatures, and homomorphic encryption [4, 5, 9, 16, 18, 21], making it vital for ensuring data confidentiality and integrity in various applications, from secure messaging to cloud computing.

The strength of the LWE problem is supported by its connections to some of the most challenging problems in lattice theory, which boosts confidence in the security of LWE-based systems. The reductions from these hard problems provide a theoretical foundation that allows cryptographers to build systems with provable security. However, this proven security often applies only within specific parameter ranges, leading to concerns about practical efficiency in real-world applications. Consequently, many implementations of LWE choose parameters that go beyond these proven ranges to enhance performance, a trade-off that must be carefully managed. A significant factor in achieving this balance is the size and structure of the secret key, which influences not only the security of the system but also its computational efficiency. This has led to the adoption of binary, ternary, or sparse vector forms in various LWE systems [7, 8, 10, 2, 6, 14, 15]. By exploring these alternatives, researchers aim to optimize both security and efficiency, ultimately enhancing the viability of LWE in practical scenarios.

Despite the development of various methods to address the LWE problem, the cryptographic security of LWE with sparse or short secret vectors remains an area of ongoing inquiry. One promising method is combinatorial attacks, which take advantage of smaller secrets to lower the complexity of attacks, especially when combined with lattice reduction techniques. Among these approaches, the meet-in-the-middle (MitM) technique introduced by Odlyzko [11] is especially relevant for LWE instances with bounded secret and error, achieving a time complexity of $S^{0.5}$, where S represents the search space size for the secret key. Notably, [3] provides an analysis focused on the case of short secret vectors.

May introduced a new combinatorial MitM attack, called MEET-LWE, specifically targeting ternary LWE instances in which both the secret and error vectors take ternary values. This approach significantly improves upon previous methods, achieving an asymptotic complexity of $S^{0.25}$ and non-asymptotic complexity of $S^{0.3}$ [17]. This innovative approach combines Howgrave-Graham’s representation technique [12] with a tree-based structure that efficiently organizes secrets and their hash values. May’s method divides the LWE secret $\mathbf{s} \in \{0, \pm 1\}^n$ into two parts, $\mathbf{s}_1$ and $\mathbf{s}_2$, in $\{0, \pm 1, \pm 2\}^n$. By making educated guesses about certain parts of the error vector $\mathbf{e}$, this method uses multiple representations of $\mathbf{s}$ to simplify the list construction process.

A recent study [13] introduces an exciting new variant called the k-Sparse Learning With Errors (k-sLWE) problem, which examines the computational challenges of using sparse coefficient matrices. Unlike traditional LWE, which uses dense matrices, k-sLWE employs random k-sparse matrices $\mathbf{A} \in \mathbb{Z}_M^{m \times n}$, where each row has exactly k non-zero elements, drawn uniformly from $\mathbb{Z}_M$. This fresh approach greatly improves efficiency in both storage and computation, as each k-sLWE sample consists of only $k + 1$ elements, resulting in a computation time of $O(k \log M)$. In comparison, standard LWE samples require $n + 1$ elements and take $O(n \log M)$ time. The parameter k is key to maintaining the difficulty of the k-sLWE problem. An interesting part of this research is the investigation of different settings for k , including constant, logarithmic, polylogarithmic, and sublinear values relative to the dimension n . The findings suggest that choosing k to be logarithmic or polylogarithmic strikes an ideal balance for cryptographic applications, offering both efficiency and strong security. While a constant k can provide

theoretical insights under certain sample constraints, it does not achieve the necessary level of security when the sample size exceeds the linear dimension.

In their work, Jain et al. raised the following open question:

“... it is completely unclear if sparse LWE with small secrets is hard.”

Motivated by this open question, we investigate the security of a bounded variant of sLWE, where both the secret and the error vectors are restricted to the set $\{-B, \dots, B\}$.

Our Contributions. We present a detailed cryptanalytic study of bounded sLWE and identify two distinct regimes determined by a sparsity threshold, which we refer to as the *threshold limit*. This threshold separates parameter settings according to the expected number of solutions of individual noisy equations.

In the first regime, where the sparsity parameter lies below the threshold limit, each noisy equation admits a sufficiently small solution space. In this case, the equations are solved independently and their corresponding secret coordinates are merged sequentially, enabling efficient recovery of the full secret vector. In the second regime, where the sparsity parameter exceeds the threshold limit, the solution space of a single noisy equation becomes too large for independent processing. To address this, we extract carefully chosen subsystems of equations such that the expected number of solutions for the involved secret variables is approximately one. We then apply a structured meet-in-the-middle merging strategy across these subsystems to reconstruct the complete secret vector. We provide explicit complexity estimates for both regimes.

We further examine how the parameter sets proposed in [13] for standard sLWE, which are designed to provide security comparable to standard LWE of dimension 1024, behave when translated to the bounded setting (see Table 2). While the original work does not claim security equivalence for the bounded variant, it is natural to compare the resulting parameters with bounded LWE of dimension 1024. Our analysis shows that, under this translation, this equivalence does not directly carry over in general, and the ternary case ($B = 1$) provides a representative example. For example, the lattice estimator [1] predicts a time complexity of approximately 2^{54} for primal lattice reduction on ternary LWE with dimension 1024 and modulus 2^{64} , whereas our analysis recovers the corresponding ternary sLWE secret in time about 2^{46} . Similarly, for moduli 2^{32} and 2^{16} , we observe a substantial reduction in the complexity estimates. These comparisons illustrate how sparsity influences the concrete security of bounded sLWE parameters.

Organization of our paper. The paper is organized as follows. In Section 2, we provide a brief introduction to the Sparse Learning With Errors (sLWE) problem. Section 3 introduces the bounded variant of sLWE. In Section 4, we present some general algorithms relevant to our approach. Sections 5 and 6 describe two algorithms designed to solve bounded sLWE instances, depending on the sparsity level. Section 7 provides the numerical results of our analysis, while Section 8 details the experimental verification of the algorithms. Finally, we conclude the paper in Section 9.

2 Preliminaries

2.1 Notation

We define $[n] = \{0, 1, 2, \dots, n-1\}$ and $[-B, B] = \{-B, \dots, B\}$. For a positive integer M , $\mathbb{Z}_M$ denotes the ring of integers modulo M . We represent $\mathbb{Z}_M$ as $\{-\lfloor \frac{M-1}{2} \rfloor, -\lfloor \frac{M-1}{2} \rfloor + 1, \dots, -1, 0, 1, \dots, \lfloor \frac{M}{2} \rfloor - 1, \lfloor \frac{M}{2} \rfloor\}$. Capital bold letters (e.g., $\mathbf{A}$) are used to denote matrices, while small bold letters (e.g., $\mathbf{v}$) represent vectors. For a vector $\mathbf{v}$ from $\mathbb{Z}_M$, we use $|\mathbf{v}|$ to denote its dimension (length); thus $|\mathbf{v}| = n$ means $\mathbf{v} \in \mathbb{Z}_M^n$. The inner product of vectors is denoted by $\langle \cdot, \cdot \rangle$. Additionally, we denote probability by $\mathbb{P}$, variance by $\mathbb{V}$, and covariance by COV .

2.2 Sparse Learning With Errors

Sparse Learning With Errors (sLWE) is a variant of the Learning With Errors (LWE) problem introduced by Jain et al. in CRYPTO 2024 [13]. We begin by formally defining this variant.

Definition 1 Let $n, m, M \in \mathbb{N}$, and let k be a constant or a polylogarithmic function of n . We consider a random matrix $\mathbf{A} \in \mathbb{Z}_M^{m \times n}$, where each row contains exactly k non-zero elements uniformly sampled from $\mathbb{Z}_M$. We refer to such matrices as k -sparse matrices. Additionally, let $\mathbf{b} \in \mathbb{Z}_M^m$. The Sparse Learning With Errors (sLWE $_{n,k,m,M,\sigma}$) problem is defined as the task of finding a secret vector $\mathbf{s} \in \mathbb{Z}_M^n$ that satisfies the equation

$$\mathbf{A}\mathbf{s} = \mathbf{b} + \mathbf{e},$$

where $\mathbf{e} \in \mathbb{Z}_M^n$ is an error vector drawn from a discrete Gaussian distribution with mean 0 and noise parameter σ .

Definition 2 For a modulus M and a vector $\mathbf{v} \in \mathbb{Z}_M^n$, the locality set of $\mathbf{v}$ is defined as the indices $S \subset [n]$ where $\mathbf{v}[i] \neq 0$ for all $i \in S$.

We demonstrate that if we have an oracle capable of solving the sparse LWE problem for dimension n and sparsity k with m samples, we can use it to solve the standard LWE problem for dimension k with the same m samples, provided the modulus p is prime.

Given an input pair $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e})$, we will transform it into a new pair $(\mathbf{A}', \mathbf{b}' = \mathbf{A}'\mathbf{s}' + \mathbf{e})$ that retains the same error properties. If $\mathbf{b}$ were sampled randomly instead of generated from the LWE distribution, this transformation would maintain that randomness. The resulting distribution of $(\mathbf{A}', \mathbf{b}')$ will match that of average-case sparse LWE. Importantly, this method does not alter the error vector $\mathbf{e}$, allowing it to also apply in scenarios like learning parity with noise.

Conversion Procedure. The conversion process works as follows: we start with a dense matrix $\mathbf{A}$ of size $m \times k$ and aim to convert each column into a corresponding sample in a new sparse matrix $\mathbf{A}'$ of size $m \times n$, where each row has exactly k non-zero entries. The vector $\mathbf{b}$ will remain unchanged for now, allowing us to express it as $\mathbf{b} = \mathbf{A}\mathbf{z}' + \mathbf{e}$. At this point, the secret $\mathbf{z}$ is not uniformly random. To address this, we can add a random term $\mathbf{A}\mathbf{v}'$ to $\mathbf{b}$ using a randomly selected vector $\mathbf{v}$. This ensures that the new secret $\mathbf{v} + \mathbf{z}$ is random and independent of $\mathbf{A}'$.

Next, we need to establish a relationship between $\mathbf{A}$ and $\mathbf{A}'$. Specifically, we want to ensure that there exists a matrix $\mathbf{C}$ of size $n \times k$ such that the equation $\mathbf{A}'\mathbf{C} = \mathbf{A}$ holds. This allows us to rewrite $\mathbf{b}$ as $\mathbf{b} = \mathbf{A}'(\mathbf{C}\mathbf{s}) + \mathbf{e}$.

To construct $\mathbf{A}'$, we first fix the matrix $\mathbf{C}$. We then randomly sample locality patterns $S_1, \dots, S_m$ from $\binom{[n]}{k}$. For each i , we select the non-zero coordinates according to S_i so that $\mathbf{A}'[i]\mathbf{C} = \mathbf{A}[i]$. This is feasible because we can view this as a system of linear equations over $\mathbb{Z}_M$, where k variables correspond to the non-zero entries of $\mathbf{A}'[i]$ and k equations relate to the columns of $\mathbf{A}[i]$. We choose $\mathbf{C}$ to be a Vandermonde matrix, ensuring that any k rows are linearly independent, which guarantees that our system is solvable.

Randomness of $\mathbf{A}'$. To confirm that $\mathbf{A}'$ is random, we note that each row $\mathbf{A}'[i]$ can be described by its locality pattern and the random non-zero entries corresponding to this pattern. Since the locality patterns are independently chosen at random, the columns of $\mathbf{A}'$ will be uniformly distributed and independent of one another.

We state the following theorem:

Theorem: [13] If there exists a probabilistic polynomial-time (PPT) algorithm that can solve average-case sLWE $_{n,k,m,p,\sigma}$ instances in time T with success probability ξ , where the modulus p is a prime and the number of samples $m < p$, then we can solve average-case LWE $_{k,m,p,\sigma}$ instances in time $T + O(n^3 m \log p)$ with the same success probability ξ .

3 Bounded Sparse Learning With Errors

In [13], the authors posed an open problem concerning the security of small-secret sparse LWE. Motivated by this question, we introduce and analyze a new variant of sLWE in which both the secret and the error are bounded. We refer to this variant as *bounded sLWE*, and formally define it below.

Definition 3 Consider dimension n , sparsity parameter k , a sample complexity m , modulus M . Let $\mathbf{A} \in \mathbb{Z}_M^{m \times n}$ be a random k -sparse matrix in which each row contains exactly k nonzero entries, and let $\mathbf{b} \in \mathbb{Z}_M^m$. A B -bounded sLWE problem ($\text{sLWE}_{n,k,m,M,B}$) asks to find a B -bounded secret vector $\mathbf{s} \in [-B, B]^n$ that holds the identity $\mathbf{As} = \mathbf{b} + \mathbf{e}$, where $\mathbf{e} \in [-B, B]^m$ is an arbitrary B -bounded vector.

Observe that, each noisy linear equation arising from a sample of a bounded $\text{sLWE}_{n,k,m,M,B}$ instance involves exactly k secret variables together with an error term. Accordingly, the search space corresponding to the variables appearing in any single equation has size $(2B+1)^{k+1}$. Under the assumption that $\mathbf{A}$ and $\mathbf{b}$ are uniformly random subject to the construction above, a uniformly chosen element of this search space satisfies a fixed equation with probability $1/M$. Therefore, the expected number of solutions for the variables occurring in a single equation is $\frac{(2B+1)^{k+1}}{M}$.

When k and B are sufficiently small, a single equation is expected to admit a unique solution with respect to the variables it involves. In this regime, the equations are independent enough to be solved separately, and the secret vector to the bounded sLWE problem can be constructed coordinate by coordinate. We formalize this notion by defining a threshold for the sparsity parameter k :

Definition 4 Given an instance of bounded $\text{sLWE}_{n,k,m,M,B}$, the *threshold limit* $\delta \in \mathbb{N}$ is defined as the largest integer such that a random equation involving at most δ variables is expected to have a unique solution.

For a random equation with δ number of B -bounded variables, the expected number of solutions modulo M is $\frac{(2B+1)^\delta}{M}$. The uniqueness of the solution implies the condition $\frac{(2B+1)^\delta}{M} \leq 1$, which leads to the threshold limit $\delta = \lfloor \log_{(2B+1)} M \rfloor$.

Now, for a noisy equation with at most δ variables from an instance of bounded $\text{sLWE}_{n,k,m,M,B}$, the noise can be interpreted as an additional variable. Consequently, the expected number of solutions of a noisy equation will be at most $\frac{(2B+1)^{\delta+1}}{M}$. Therefore, for instances where the sparsity parameter $k < \delta$, each equation is expected to have a unique solution. When $k = \delta$, each equation is expected to have at most $(2B+1)$ solutions. In this case, it is likely that selecting a constant number of random equations is sufficient to provide a unique solution, provided the equations will share some variables. Hence, when $k \leq \delta$, one solves each equation and iteratively combines the solutions to recover the secret vector.

However, when k exceeds the threshold limit δ , solving the entire system of equations directly becomes computationally infeasible due to the exponential growth of the search space. To address this, we propose decomposing the system into smaller subsystems, where each subsystem can be solved independently.

Consider a subsystem with μ equations and v variables. The search space for the solutions corresponding to the present variables of such a subsystem is $(2B+1)^{v+\mu}$, as there are v number of B -bounded variables and μ number of B -bounded errors to account for. The probability that a randomly chosen element from this search space satisfies all μ equations is $\frac{1}{M^\mu}$. Hence, the expected number of solutions for the subsystem is $\frac{(2B+1)^{v+\mu}}{M^\mu}$.

If we can extract and solve smaller subsystems efficiently, the solutions to these subsystems can be merged to recover the full secret vector $\mathbf{s}$. This decomposition approach utilizes the sparsity and structure of the problem, allowing us to tackle larger instances of B -bounded sLWE.

Consider a sample $(\mathbf{a}, b)$ from a bounded $\text{sLWE}_{n,k,m,M,B}$ instance. The vector $\mathbf{a}$ is k -sparse, meaning it contains exactly k non-zero entries. In the following sections, we denote by $\text{idx}_{\mathbf{a}}$ the locality set of $\mathbf{a}$, which is the set of indices corresponding to the non-zero elements of $\mathbf{a}$. We also write $\mathbf{a}|_{\text{idx}_{\mathbf{a}}}$ to denote the projection of $\mathbf{a}$ onto the index set $\text{idx}_{\mathbf{a}}$.

Example 1 Consider a 3-sparse vector $\mathbf{a} = (0, 0, 7, 0, 3, 5, 0) \in \mathbb{Z}_{16}^7$. In this case, $\text{idx}_{\mathbf{a}} = \{2, 4, 5\}$, which corresponds to the positions of the non-zero entries, and the projected vector is $\mathbf{a}|_{\text{idx}_{\mathbf{a}}} = (7, 3, 5)$.

4 General Algorithms

In this section, we will explore three algorithms essential for solving the bounded sLWE problems.

4.1 Algorithm: Get_Subsystem

We begin by describing the **Get_Subsystem Algorithm**, which extracts a subsystem consisting of at most v variables and exactly μ equations from a given set of samples of a bounded sLWE instance. The algorithm is straightforward: it randomly selects μ samples from the sLWE instance and checks whether the selected samples involve no more than v distinct variables. If this condition is met, the selected samples are returned as a valid subsystem. The complete procedure is detailed in Algorithm 1.

Algorithm 1: Get_Subsystem

Input: v, μ , Samples of a bounded sLWE $_{n,k,m,M,B}$ instance
Output: A subsystem containing at most v variables and μ equations.
1 repeat
2 | $S \leftarrow$ randomly chosen μ samples from sLWE instance
3 until;
4 S contains at most v variables
5 return S ;

Algorithm 1 identifies a subsystem with at most v variables and μ equations from a given bounded sLWE instance. The algorithm's time complexity is dependent on how many times μ samples need to be randomly chosen to ensure the subsystem contains at most v variables. Let $\tilde{p}$ represent the probability of finding such a subsystem. To analyze this probability, we present the following theorem:

Theorem 1 Let $A_0, A_1, \dots, A_{\mu-1}$ be random subsets of the set $[n]$ such that $|A_i| = k_i$ for each $i = 0, 1, \dots, \mu - 1$. Define the random variable X as the size of the union $A_0 \cup A_1 \cup \dots \cup A_{\mu-1}$. Then, the expected value of X is given by

$$\mathbb{E}(X) = n \left[1 - \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right) \right],$$

and the variance of X can be expressed as:

$$\mathbb{V}(X) = n \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right) - n^2 \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right)^2 + n(n-1) \prod_{j=0}^{\mu-1} \left(1 - \frac{2k_j}{n} + \frac{k_j(k_j-1)}{n(n-1)} \right).$$

Proof The proof is provided in Appendix A.

From the above theorem, we have the following corollary:

Corollary 1 Let $A_0, A_1, \dots, A_{\mu-1}$ be random subsets of the set $[n]$ such that $|A_i| = k$ for each $i = 0, 1, \dots, \mu - 1$. Define the random variable X as the size of the union $A_0 \cup A_1 \cup \dots \cup A_{\mu-1}$. Then, the expected value of X is given by

$$\mathbb{E}(X) = n \left[1 - \left(1 - \frac{k}{n} \right)^\mu \right],$$

and the variance of X can be expressed as:

$$\mathbb{V}(X) = n \left(1 - \frac{k}{n} \right)^\mu - n^2 \left(1 - \frac{k}{n} \right)^{2\mu} + n(n-1) \left[1 - \frac{2k}{n} + \frac{k(k-1)}{n(n-1)} \right]^\mu.$$

For sufficiently large n , the distribution of X can be well approximated by a normal distribution with the mean and variance given above, in accordance with the Central Limit Theorem. This approximation enables estimation of the probability that μ randomly selected subsets collectively involve at most v variables, namely $\mathbb{P}(X \leq v)$. Consequently, this probability can be expressed as $\mathbb{P}\left(\frac{v - \text{mean}}{\text{standard deviation}} \leq 1\right)$.

Using the above result, we can estimate the probability $\tilde{p}$ of selecting a subsystem consisting of μ equations involving at most v variables. Consequently, in order to obtain such a subsystem, the selection process must be repeated approximately $\mathcal{O}\left(\frac{1}{\tilde{p}}\right)$ times in expectation. Therefore, the overall time complexity of Algorithm 1 is given by:

$$\mathcal{T}_{\text{get_subsystem}} = \mathcal{O}\left(\frac{1}{\tilde{p}}\right).$$

4.2 Algorithm: Solve

Consider an equation of the form

$$\langle \mathbf{a}, \mathbf{x} \rangle = b + e,$$

where the unknown vector $\mathbf{x} \in [-B, B]^\omega$, the coefficient vector $\mathbf{a} \in \mathbb{Z}_M^\omega$, and the error term $e \in [-B, B]$. To solve this equation, we adopt a meet-in-the-middle (MitM) strategy. We partition the variables and the coefficient vector into two blocks,

$$\mathbf{x} = (\mathbf{x}_1 \parallel \mathbf{x}_2) \in \mathbb{Z}_M^\kappa \times \mathbb{Z}_M^{\omega-\kappa}, \quad \mathbf{a} = (\mathbf{a}_1 \parallel \mathbf{a}_2) \in \mathbb{Z}_M^\kappa \times \mathbb{Z}_M^{\omega-\kappa},$$

where, $\kappa = \lceil \frac{\omega+1}{2} \rceil$ and rewrite the equation as

$$\langle \mathbf{a}_1, \mathbf{x}_1 \rangle = b + e - \langle \mathbf{a}_2, \mathbf{x}_2 \rangle.$$

To efficiently search for solutions, we first compute the set of all possible inner products for the first κ terms:

$$T_{\text{left}} = \{(\mathbf{y}_1, \langle \mathbf{a}_1, \mathbf{y}_1 \rangle) : \mathbf{y}_1 \in [-B, B]^\kappa\}.$$

Next, for the remaining $\omega - \kappa$ terms, including the error term e , we compute:

$$T_{\text{right}} = \{(\mathbf{y}_2, b + e - \langle \mathbf{a}_2, \mathbf{y}_2 \rangle) : (\mathbf{y}_2, e) \in [-B, B]^{\omega-\kappa+1}\}.$$

Any collision between the second coordinates of T_{left} and T_{right} corresponds to a valid solution of the original equation.

The **Solve Algorithm** takes $\mathbf{a}, b$ and index set idx as input such that $|\mathbf{a}| = |\text{idx}|$ and then solves the equation

$$\langle \mathbf{a}, \mathbf{x} \rangle = b + e$$

using the above described process. For each recovered solution of the equation, the algorithm constructs a vector $\mathbf{y} \in [-B, B]^n$ by placing the coordinates of the solution at the positions specified by idx and setting all remaining coordinates to zero. All such vectors $\mathbf{y}$ are stored in an output table.

Algorithm 2: Solve

Input: $\mathbf{a}, b, \text{idx}$
Output: A table containing B -bounded solutions for the involved variables in $\langle \mathbf{a}, \mathbf{x} \rangle = b + e$

```

1  $\omega \leftarrow |\mathbf{a}|, \kappa \leftarrow \lceil \frac{\omega+1}{2} \rceil$ 
2  $T \leftarrow \emptyset$ 
3  $(\mathbf{a}_1 \parallel \mathbf{a}_2) \leftarrow \mathbf{a}$ , where  $|\mathbf{a}_1| = \kappa$  and  $|\mathbf{a}_2| = \omega - \kappa$ 
4  $T_{\text{left}} \leftarrow \{(\mathbf{y}_1, \langle \mathbf{a}_1, \mathbf{y}_1 \rangle) : \mathbf{y}_1 \in [-B, B]^\kappa\}$ 
5  $T_{\text{right}} \leftarrow \{(\mathbf{y}_2, b + e - \langle \mathbf{a}_2, \mathbf{y}_2 \rangle) : (\mathbf{y}_2, e) \in [-B, B]^{\omega-\kappa+1}\}$ 
   // Collision search via MitM
6 for  $(\mathbf{y}_1, z_1) \in T_{\text{left}}$  and  $(\mathbf{y}_2, z_2) \in T_{\text{right}}$  do
7   if  $z_1 = z_2$  then
8      $\mathbf{y} = (0, 0, \dots, 0)$ ; // Initialize zero vector of length  $n$ 
9     for  $i = 0$  to  $\omega - 1$  do
10       $\mathbf{y}_{\text{idx}[i]} \leftarrow (\mathbf{y}_1 \parallel \mathbf{y}_2)_i$ 
11     $T \leftarrow T \cup \{\mathbf{y}\}$ 
12 return  $T$ ;
```

Example 2 For an example, consider a sample $(\mathbf{a}', b)$ from the bounded sLWE $_{n,k,m,M,B}$ instance, the vector $\mathbf{a}'$ is k -sparse. Let $\mathbf{a}$ be the projection of $\mathbf{a}'$ onto the index set $\text{idx}_{\mathbf{a}'}$ corresponding to the vector $\mathbf{a}'$ and $\text{idx} = \text{idx}_{\mathbf{a}'}$. The **Solve Algorithm** takes $\mathbf{a}, b$ and idx as input and then solves the equation

$$\langle \mathbf{a}, \mathbf{x} \rangle = b + e$$

for the variables corresponding to the index set idx .

The time complexity of Algorithm 2 is dominated by the construction of the lists T_{left} and T_{right} , together with the subsequent collision search between them. The sizes of these lists are

$$|T_{\text{left}}| = (2B + 1)^\kappa, \quad |T_{\text{right}}| = (2B + 1)^{\omega-\kappa+1}.$$

The expected number of collisions, which is equal to the expected number of solutions to the equation, is

$$N_{\text{col}} = \frac{(2B + 1)^{\omega+1}}{M}.$$

Consequently, the time complexity of the **Solve Algorithm** is

$$\mathcal{T}_{\text{solve}} = \mathcal{O}(\max\{|T_{\text{left}}|, |T_{\text{right}}|, N_{\text{col}}\}) = \mathcal{O}\left(\max\left\{(2B + 1)^\kappa, \frac{(2B + 1)^{\omega+1}}{M}\right\}\right),$$

and the memory complexity satisfies

$$\mathcal{M}_{\text{solve}} = \mathcal{O}(\max\{|T_{\text{left}}|, |T_{\text{right}}|, N_{\text{col}}\}) = \mathcal{O}\left(\max\left\{(2B + 1)^\kappa, \frac{(2B + 1)^{\omega+1}}{M}\right\}\right).$$

4.3 Algorithm: Merge

Now, let us discuss the **Merge Algorithm**. Consider two pairs of tables and corresponding index sets: (T_1, idx_1) and (T_2, idx_2) . Each table T_i contains a set of vectors, where every vector holds possible values for the B -bounded secret vector restricted to the index set idx_i , with entries corresponding to indices outside idx_i set to zero, obtained by solving some equations from the bounded sLWE $_{n,k,m,M,B}$ samples.

Algorithm 3: Merge

Input: $(T_1, \text{idx}_1), (T_2, \text{idx}_2)$
Output: Merged vectors from T_1 and T_2 if they share the same coordinates in $\text{idx}_1 \cap \text{idx}_2$

```

1 if  $\text{idx}_1 = \emptyset$  then
2   return  $T_2, \text{idx}_2$ 
3 else if  $\text{idx}_2 = \emptyset$  then
4   return  $T_1, \text{idx}_1$ 
5 else
6    $\text{idx} \leftarrow \text{idx}_1 \cap \text{idx}_2$  // common indices
7    $T \leftarrow \emptyset$ ;
8    $T_{\text{left}} \leftarrow \{(\mathbf{x}, \mathbf{x}|_{\text{idx}}) : \mathbf{x} \in T_1\}$ 
9    $T_{\text{right}} \leftarrow \{(\mathbf{x}, \mathbf{x}|_{\text{idx}}) : \mathbf{x} \in T_2\}$ 
   // Collision search via MitM
10  for  $(\mathbf{x}, \mathbf{x}') \in T_{\text{left}}$  and  $(\mathbf{y}, \mathbf{y}') \in T_{\text{right}}$  do
11    if  $\mathbf{x}' = \mathbf{y}'$  then
12       $\mathbf{z} = (0, 0, \dots, 0)$ ; // Initialize zero vector of length  $n$ 
13      for  $i \in \text{idx}_1$  do
14         $\mathbf{z}_i \leftarrow \mathbf{x}_i$ 
15      for  $i \in \text{idx}_2 \setminus \text{idx}_1$  do
16         $\mathbf{z}_i \leftarrow \mathbf{y}_i$ 
17       $T \leftarrow T \cup \{\mathbf{z}\}$ 
18  return  $T, \text{idx}_1 \cup \text{idx}_2$ ;

```

Our goal is to construct a new set of vectors representing possible values of the secret vector restricted to the union of the index sets, $\text{idx}_1 \cup \text{idx}_2$. This requires merging vectors from T_1 and T_2 under the condition that they agree on the common indices, $\text{idx}_1 \cap \text{idx}_2$.

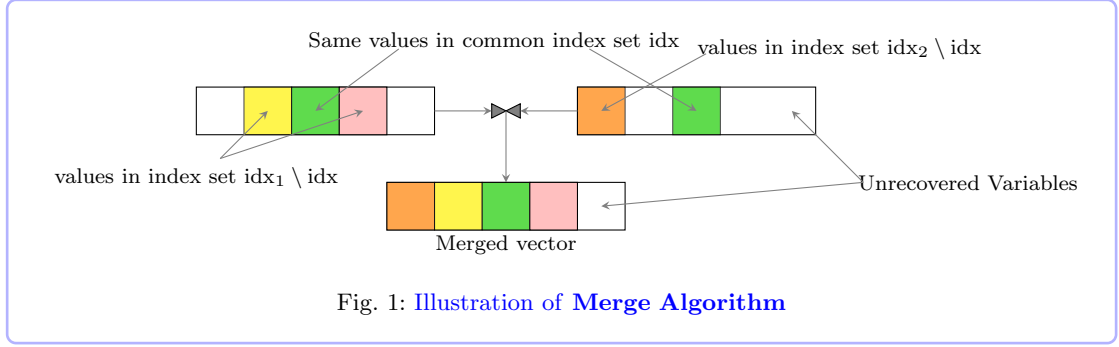
The **Merge Algorithm** achieves this by following these key steps:

1. We define two new tables, T_{left} and T_{right} , based on T_1 and T_2 , respectively. Each entry in these tables consists of a vector paired with its projection onto the common index set $\text{idx} = \text{idx}_1 \cap \text{idx}_2$. Specifically, for T_{left} , we pair each vector of T_1 with its projection onto $\text{idx}_1 \cap \text{idx}_2$, and similarly for T_{right} .
2. The algorithm then compares the pair of vectors from T_{left} and T_{right} using MitM fashion. If a pair of vectors has identical projections on the common index set $\text{idx}_1 \cap \text{idx}_2$, the vectors are merged by:
 - (a) Initializing a zero vector $\mathbf{z}$ of size n .
 - (b) Copying the components of the vector from T_{left} corresponding to idx_1 .
 - (c) Filling the remaining components from the vector in T_{right} corresponding to $\text{idx}_2 \setminus \text{idx}_1$.
3. The resulting merged vector $\mathbf{z}$ is added to the final output set T .
4. Finally, the algorithm returns the set of merged vectors T and the combined index set $\text{idx}_1 \cup \text{idx}_2$.

An illustration of the algorithm is provided in Figure 1.

Similar to the **Solve Algorithm**, the time complexity of Algorithm 3 is dominated by the construction of the lists T_{left} and T_{right} , as well as the total number of collisions. Let the input lists T_1 and T_2 for Algorithm 3 be created as follows: T_1 contains all possible values of variables corresponding to the index set idx_1 , which is obtained by solving μ_1 number of equations, while T_2 contains all possible values of variables corresponding to the index set idx_2 , obtained by solving μ_2 number of equations of a bounded sLWE $_{n,k,m,M,B}$ instance.

The number of collisions, i.e., the number of merged vectors from T_1 and T_2 that share the same coordinates in $\text{idx}_1 \cap \text{idx}_2$, is equivalent to the total number of solutions of those $\mu_1 + \mu_2$ equations.



Therefore, the size of collisions is given by:

$$|T| = \frac{(2B+1)^{v+\mu_1+\mu_2}}{M^{\mu_1+\mu_2}},$$

where $v = |\text{idx}_1 \cup \text{idx}_2|$. Thus, the time complexity of the **Merge Algorithm** is

$$\mathcal{T}_{\text{merge}} = \mathcal{O}(\max\{|T_{\text{left}}|, |T_{\text{right}}|, |T|\})$$

and the memory complexity is

$$\mathcal{M}_{\text{merge}} = \mathcal{O}(\max\{|T_{\text{left}}|, |T_{\text{right}}|, |T|\}).$$

5 Solving Bounded sLWE with Sparsity at most the Threshold Limit

Consider an instance of the B -bounded sLWE problem with dimension n , sample complexity m , modulus M , and a sparsity parameter k that is less than or equals the threshold limit $\delta = \lfloor \log_{(2B+1)} M \rfloor$. In this case, we will select a subsystem S with μ equations from the samples such that they involve all the variables of the secret vector $\mathbf{s}$. The construction of the set S is guided by a greedy heuristic aimed at efficiently recovering the full support of the secret vector. The process is initiated by selecting an arbitrary sample $(\mathbf{a}, b)$ from the bounded sLWE instance. Subsequently, S is augmented in successive steps by incorporating samples that exhibit minimal overlap in variables with those already contained in S . This iterative procedure is continued until the union of variables in S spans the entire set of n coordinates of the secret.

Since the sparsity parameter is less than or equals the threshold limit, we expect that these equations will have a constant number of solutions corresponding to the present variables, with a maximum expected solution count of $2B+1$. Once these equations are identified, we rearrange the equations based on **maximal variable overlap condition** in the following way: First, we will place the 0^{th} equation in the 0^{th} position of the reordered set. Then, we will select an equation from the remaining set with maximal variable overlap with the 0^{th} equation. After that, for the 2^{nd} position, we will select an equation that has the highest variable overlap with the union of variables present in the already selected equations (i.e., the 0^{th} and 1^{st} equations). We will continue this process until all equations are arranged.

We solve the rearranged equations iteratively using Algorithm 2 and merge the solutions using Algorithm 3 to obtain every coordinate of the secret vector. This overall process can be found in Algorithm 4, and the illustration can be found in Figure 2.

Analytical Upper Bound of μ . In the following, we present an approach that provides an analytical upper bound on μ . To analyze the expected number of samples μ required for S to recover the full support of the secret vector, we adopt a stage-wise probabilistic framework. Suppose at i^{th} stage, S includes v

Algorithm 4: Algorithm for Solving Bounded sLWE _{n,k,m,M,B} Instances with Sparsity Less Than or Equal to the Threshold

Input: $(\mathbf{A}, \mathbf{b}) \in \mathbb{Z}_M^{m \times n} \times \mathbb{Z}_M^m$, sparsity $k \in \mathbb{N}$
Output: Secret vector $\mathbf{s} \in [-B, B]^n$ such that $\mathbf{e} = \mathbf{A}\mathbf{s} - \mathbf{b} \bmod M$ is B -bounded
// Find a subsystem S covering all secret variables

- 1 $S \leftarrow$ a random sample $(\mathbf{a}, b)$ from the sLWE instance
- 2 **repeat**
- 3 Find a sample $(\mathbf{a}, b)$ from the sLWE instance with minimal variable overlap with S ;
- 4 $S \leftarrow S \cup \{(\mathbf{a}, b)\}$
- 5 **until**;
- 6 S has exactly n variables
- 7 $\mu \leftarrow |S|$
- 8 *// Rearrange S w.r.t maximal variable overlap condition*
- 8 $I \leftarrow \{0\}$, $V \leftarrow$ set of variables of the sample $S[0]$
- 9 **while** $|I| < \mu$ **do**
- 10 Find $i \in [\mu] \setminus I$ such that $S[i]$ has maximal variable overlap with V ;
- 11 $I \leftarrow I \cup \{i\}$
- 12 $V \leftarrow V \cup$ set of variables of $S[i]$
- 13 Rearrange S with respect to I ;
- 14 *// Solve the subsystem S*
- 14 $T = \emptyset, \text{idx} = \emptyset$
- 15 **for** $(\mathbf{a}, b) \in S$ **do**
- 16 $\mathbf{a}' \leftarrow \mathbf{a}_{|\text{idx}_{\mathbf{a}}|}$
- 17 $T_{\mathbf{a}} \leftarrow \text{Solve}(\mathbf{a}', b, \text{id}_{\mathbf{a}})$
- 18 $T, \text{idx} \leftarrow \text{Merge}((T, \text{idx}), (T_{\mathbf{a}}, \text{id}_{\mathbf{a}}))$
- 19 **if** $|T| = 1$ **then**
- 20 **return** $\mathbf{x} \in T$
- 21 **else**
- 22 **for** $\mathbf{x} \in T$ **do**
- 23 **if** $\mathbf{A}\mathbf{x} - \mathbf{b} \in [-B, B]^m$ **then**
- 24 **return** $\mathbf{x}$

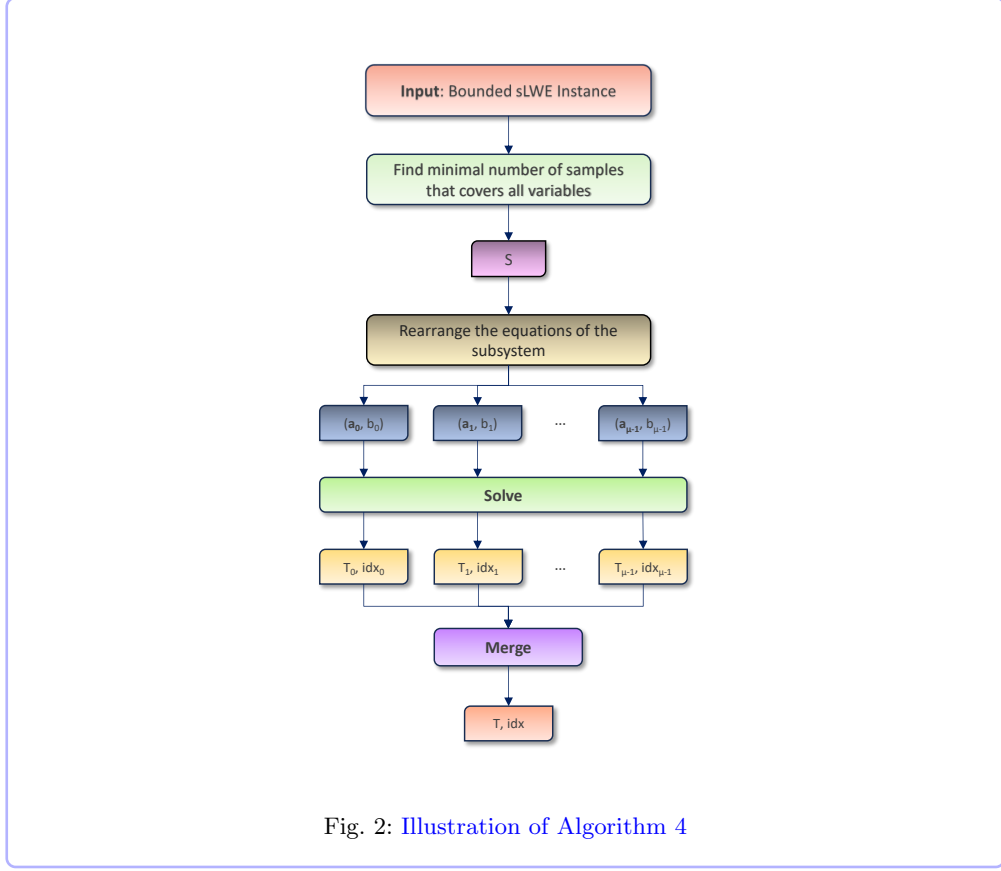
unique variables and contains i samples. Consider an arbitrary sample $(\mathbf{a}, b)$ from the remaining pool. Let X be a random variable representing the size of the union of the variables in S and the variables in the sample $(\mathbf{a}, b)$. From Theorem 1, we know that the expected value of X is given by:

$$\bar{X} := \mathbb{E}(X) = n - n \left(1 - \frac{v}{n}\right) \left(1 - \frac{k}{n}\right)$$

The variance of X is:

$$\mathbb{V}(X) = n \left(1 - \frac{v}{n}\right) \left(1 - \frac{k}{n}\right) - n^2 \left(1 - \frac{v}{n}\right)^2 \left(1 - \frac{k}{n}\right)^2 + n(n-1) \left(1 - \frac{2v}{n} + \frac{v(v-1)}{n(n-1)}\right) \left(1 - \frac{2k}{n} + \frac{k(k-1)}{n(n-1)}\right)$$

Using this expected value $\bar{X}$ and the standard deviation $\sigma = \sqrt{\mathbb{V}(X)}$, we can compute the probability of obtaining a union size v_1 that exceeds $\bar{X} + \alpha\sigma$. Let $\tilde{p} = \mathbb{P}(X \geq \lceil \bar{X} + \alpha\sigma \rceil)$ represent this probability. We choose α such that $\frac{1}{p} \leq m$ for all stages except the last few, ensuring that there exists a sample which, when added to the subsystem, will result in a subsystem with $v_1 \geq \lceil \bar{X} + \alpha\sigma \rceil$ variables. Since we are using a greedy algorithm, the algorithm will always find such a sample. In the subsequent stage, we



can consider the number of variables in the subsystem to be $\lceil \bar{X} + \alpha\sigma \rceil$ in the worst-case scenario, and compute the mean and variance accordingly, continuing in the same manner as before.

If at the $(i + 1)^{\text{th}}$ stage $\frac{1}{p} > m$ and the subsystem contains v many variables, we can conclude that to cover the remaining $n - v$ variables, we will need at most $n - v$ additional samples. Therefore, the size of the subsystem constructed by the greedy algorithm can be upper-bounded by $\mu = i + (n - v)$. Otherwise, if the condition $\frac{1}{p} \leq m$ holds at every stage, then μ is defined as the final stage at which all variables are covered.

Table 1 compares the experimental value of μ , computed as the average over 1000 experiments, with the theoretical upper bound obtained from the above analysis, using the parameter sets listed in Table 4. Since the algorithm's complexity depends on $\log \mu$, the close alignment of the logarithms of the experimental values and theoretical upper bounds in Table 1 indicates the consistency of the analysis.

A different analysis approach. An alternative method for deriving the expected number of samples required to cover the full support of the secret vector, using greedy selection, is presented in Appendix C.

Complexity Analysis. The time complexity of Algorithm 4 is primarily determined by the process of finding μ equations that involve exactly n variables and solving them. The time complexity to find such a subsystem using the greedy algorithm is $\mathcal{O}(\mu m)$.

The time complexity to arrange the equations is $\mathcal{O}(\mu^2)$, which is negligible. Now, from the analysis of the **Solve Algorithm**, solving the μ equations requires a time and memory of $\mathcal{O}(\mu(2B + 1)^\kappa)$, and the memory required is $\mathcal{O}((2B + 1)^\kappa)$, where $\kappa = \lceil \frac{k+1}{2} \rceil$.

Table 1: Comparison between experimental value (Exp. val) and theoretical upper bound (Theo. UB) of μ

k	$m = 2^{13}$				$m = 2^{17}$				$m = 2^{21}$			
	n	Exp. val of μ	α	Theo. UB	n	Exp. val of μ	α	Theo. UB	n	Exp. val of μ	α	Theo. UB
20	1218	98.78	2	127	1425	101.80	3	125	1656	109.05	4	132
30	1143	68	3	76	1265	66.71	3	84	1395	67.84	4	84
40	1110	53.09	3	60	1195	50.96	3	64	1285	50.61	4	63

If the sparsity parameter satisfies $k < \delta$, each equation is expected to admit a unique solution for the variables involved. In the boundary case $k = \delta$, as discussed earlier, each equation is expected to admit at most $2B + 1$ solutions. By combining the solutions from a constant number of such equations, a unique assignment for the involved variables can be obtained. Since the equations are arranged to ensure overlapping variable support, the merging process produces only a constant number of candidate vectors. As a result, the runtime of the **Merge Algorithm** in this regime is dominated by the list construction phase and is therefore $\mathcal{O}(\mu(2B + 1)^\kappa)$.

Hence the overall time complexity to solve a bounded sLWE instance with sparsity parameter less than or equal to the threshold limit is given by:

$$\mathcal{O}(\max\{\mu m, \mu(2B + 1)^\kappa\}),$$

and as the equations are solved and merged iteratively, the memory complexity becomes:

$$\mathcal{O}(\max\{m, (2B + 1)^\kappa\}).$$

Remark 1 When the sparsity parameter k is less than or equal to the threshold limit δ , each equation is expected to yield at most $2B + 1$ solutions corresponding to the present variables. In such cases, instead of using the naive MitM approach described in Algorithm 2, one can employ the Schroeppel-Shamir [20] variant of MitM, which achieves the same runtime but uses only the square root of the memory. However, for the parameter settings listed in Table 4, our memory requirements are already within a practical range. Therefore, we use the Algorithm 2 for simplicity.

6 Solving Bounded sLWE with Sparsity Above the Threshold Limit

Consider an instance of the bounded sLWE problem with dimension n , sample complexity m , modulus M , and a sparsity parameter k that exceeds the threshold limit $\delta = \lfloor \log_{(2B+1)} M \rfloor$. As mentioned earlier, solving the entire system of equations iteratively is not feasible due to the exponential growth in the number of solutions. Thus, we focus on solving smaller subsystems and merging their solutions progressively, rather than tackling the entire instance at once.

As opposed to the case of sparsity parameter below the threshold limit, we are initially searching for subsystems that cover a minimum number of variables. The process begins by selecting r subsystems of the bounded sLWE instance, each consisting of at most v variables and exactly μ equations, where v, μ and r are optimization parameters. Such subsystems can be found using Algorithm 1.

We choose optimal r in such a way that

1. We require that the combined subsystem of all r subsystems is expected to admit a unique solution corresponding to the variables present in the combined subsystem. From Theorem 1, the expected number of variables in the combined subsystem is $v' = n \left[1 - \left(1 - \frac{v}{n} \right)^r \right]$, and the number of equations is $r\mu$. Thus, the expected number of solutions in the combined subsystem is $\frac{(2B+1)^{v'+r\mu}}{M^{r\mu}}$, and we

Algorithm 5: Algorithm for Solving Bounded sLWE _{n,k,m,M,B} Instances with Sparsity Above the Threshold

Input: $(\mathbf{A}, \mathbf{b}) \in \mathbb{Z}_M^{m \times n} \times \mathbb{Z}_M^m$, sparsity $k \in \mathbb{N}$
Output: Secret vector $\mathbf{s} \in [-B, B]^n$ such that $\mathbf{e} = \mathbf{A}\mathbf{s} - \mathbf{b} \bmod M$ is B -bounded

- 1 $\delta \leftarrow \lfloor \log_{(2B+1)} M \rfloor$
- 2 Choose optimal v, μ, r ;
 // Find r subsystems consisting of μ equations and at most v variables
- 3 **for** $j = 0$ **to** $r - 1$ **do**
- 4 $S_j \leftarrow \text{Get_Subsystem}(v, \mu, \text{samples of sLWE})$
- 5 Remove S_j from the samples of the sLWE instance;
 // Rearrange the subsystems w.r.t maximal variable overlap condition
- 6 $I \leftarrow \{0\}$, $V \leftarrow$ set of variables of S_0
- 7 **while** $|I| < r$ **do**
- 8 Find $i \in [r] \setminus I$ such that S_i has maximal variable overlap with V ;
- 9 $I \leftarrow I \cup \{i\}$
- 10 $V \leftarrow V \cup$ set of variables of S_i
- 11 Rearrange the subsystems with respect to I ;
 // Solve and Merge the subsystems
- 12 $T \leftarrow \emptyset, \text{idx} \leftarrow \emptyset$ **for** $j = 0$ **to** $r - 1$ **do**
- 13 // Rearrange S_j w.r.t maximal variable overlap condition
 $I \leftarrow \{0\}$, $V \leftarrow$ set of variables of $S_j[0]$
- 14 **while** $|I| < \mu$ **do**
- 15 Find $i \in [\mu] \setminus I$ such that $S_j[i]$ has maximal variable overlap with V ;
- 16 $I \leftarrow I \cup \{i\}$
- 17 $V \leftarrow V \cup$ set of variables of $S_j[i]$
- 18 Rearrange the equations of S_j with respect to I ;
 // Solve the subsystem S_j
- 19 $T_j \leftarrow \emptyset, \text{idx}_j \leftarrow \emptyset$
- 20 **for** $(\mathbf{a}, b) \in S_j$ **do**
- 21 $\mathbf{a}' \leftarrow \mathbf{a}|_{\text{idx}_{\mathbf{a}}}$
- 22 $T_{\mathbf{a}} \leftarrow \text{Solve}(\mathbf{a}', b, \text{idx}_{\mathbf{a}})$
- 23 $T_j, \text{idx}_j \leftarrow \text{Merge}((T_j, \text{idx}_j), (T_{\mathbf{a}}, \text{idx}_{\mathbf{a}}))$
- 24 $T, \text{idx} \leftarrow \text{Merge}((T, \text{idx}), (T_j, \text{idx}_j))$
 // Recover the remaining secret variables
- 25 **while** $|\text{idx}| < n$ **do**
- 26 Choose a sample $(\mathbf{a}, b)$ from the rest of the instance with minimal variable overlap and
 $|\text{idx}_{\mathbf{a}} \setminus \text{idx}| < \delta$;
- 27 Substitute the known variable values into the equation to obtain the form: $\langle \mathbf{a}', x \rangle = b' + e$
 where, $|\mathbf{a}'| = |\text{idx}_{\mathbf{a}} \setminus \text{idx}|$;
- 28 $\text{idx}_{\mathbf{a}'} \leftarrow \text{idx}_{\mathbf{a}} \setminus \text{idx}$
- 29 $T_{\mathbf{a}'} \leftarrow \text{Solve}(\mathbf{a}', b', \text{idx}_{\mathbf{a}'})$
- 30 $T, \text{idx} \leftarrow \text{Merge}((T, \text{idx}), (T_{\mathbf{a}'}, \text{idx}_{\mathbf{a}'}))$
- 31 **return** $\mathbf{x} \in T$

impose the condition that this value should be not larger than 1 to ensure the uniqueness of the solution. Specifically, we require:

$$(2B + 1)^{v' + r\mu} \leq M^{r\mu}.$$

2. For any randomly chosen sample from the remaining system, the number of non-overlapping variables with the combined subsystem present in the sample must be less than δ .

From Theorem 1, expected number of total variables in the combined subsystem and the sample is

$$n \left[1 - \left(1 - \frac{v'}{n} \right) \left(1 - \frac{k}{n} \right) \right].$$

So, the expected number of non-overlapping variables with the combined subsystem present in the new sample is given by:

$$n \left[1 - \left(1 - \frac{v'}{n} \right) \left(1 - \frac{k}{n} \right) \right] - v'$$

To ensure the condition, we require:

$$n \left[1 - \left(1 - \frac{v'}{n} \right) \left(1 - \frac{k}{n} \right) \right] - v' < \delta$$

Once r such subsystems are selected from the sLWE instance, we rearrange them based on **maximal variable overlap condition**. Specifically, we begin by placing the 0^{th} subsystem in the 0^{th} position. Next, we select the subsystem that shares the maximal number of variables with the 0^{th} and place it in the 1^{st} position. Then, we choose the subsystem that has the maximal variable overlap with the union of variables in the already placed 0^{th} and 1^{st} subsystems and place it in the 2^{nd} position. This process continues until all subsystems are arranged.

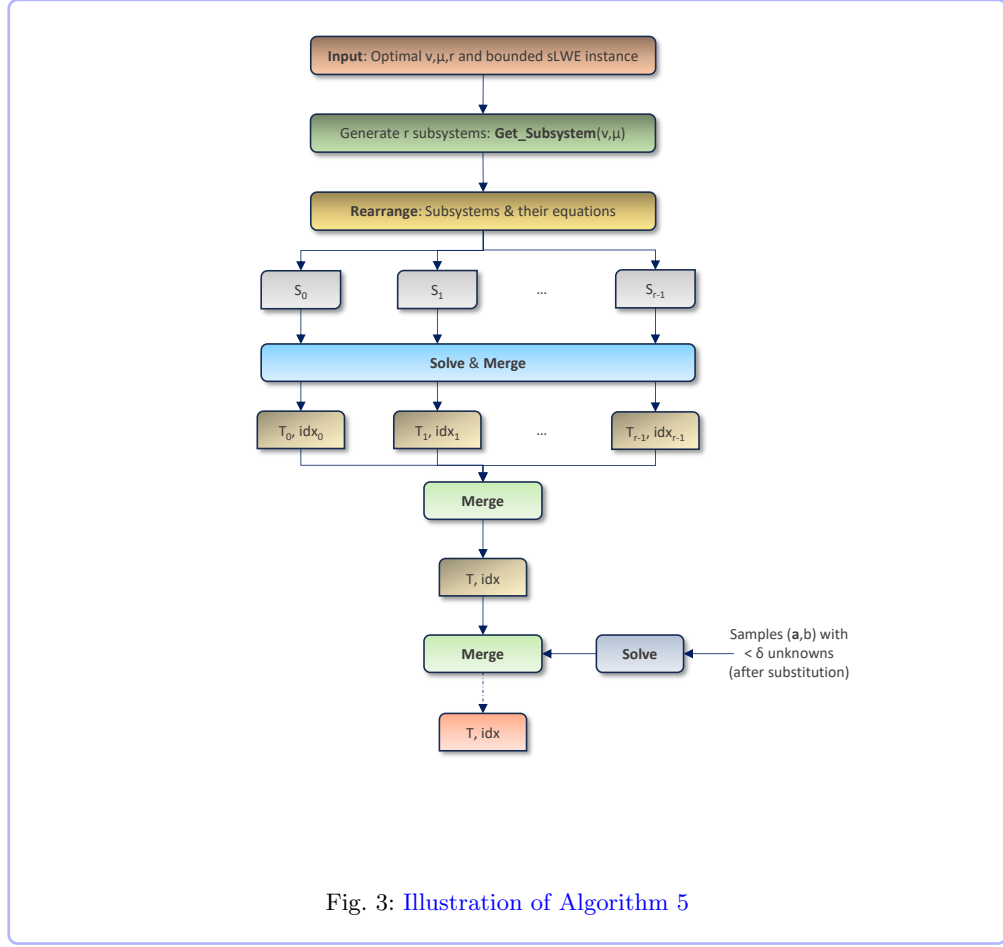
After this, we apply the same strategy within each subsystem: rearranging their individual equations according to maximal variable overlap condition, starting from the first equation and iteratively selecting the next one based on maximal overlap with the variables in the already placed equations.

After these rearrangements we then solve every subsystem iteratively. Afterward, we merge their solutions using Algorithm 3 to obtain the solution of the combined subsystem. The first condition on r ensures that the combined subsystem is expected to have a unique solution for the present variables. We then select additional samples from the remaining instances so that the number of variables not overlapping with the combined subsystem is maximized but remains below δ , enabling recovery of the remaining variables of the secret vector. The second condition on r ensures that, with high probability, we can find such samples. We then substitute the values of the known variables and solve using the Algorithm 2. After the substitution, the number of unknown variables in the sample will be at most $\delta - 1$, which implies that the sample will have an expected unique solution. Afterward, the solution to the combined subsystem (consisting of both the combined subsystem and the new sample) is obtained using Algorithm 3. This process continues iteratively until all coordinates of the secret vector are recovered. The entire procedure is summarized in Algorithm 5, and the illustration is presented in Figure 3.

Complexity Analysis. The overall time complexity of Algorithm 5 is determined by three factors: (1) the search for r subsystems, each containing at most v variables and μ equations, (2) solving each subsystem, and (3) merge the solutions of the subsystems.

Let $\tilde{p}$ represent the probability of finding a subsystem with at most v variables and μ equations, which can be derived from Normal distribution using mean and variance from Theorem 1. As discussed in the analysis of the **Get_Subsystem Algorithm**, locating such a subsystem requires checking $\mathcal{O}(\frac{1}{\tilde{p}})$ subsystems with μ equations. In Algorithm 5, we need to identify r such subsystems. Consequently, we must iterate through $\mathcal{O}(\frac{r}{\tilde{p}})$ subsystems. Thus, the time complexity for finding r subsystems, each with at most v variables and μ equations, is given by:

$$\mathcal{O}(r\mathcal{T}_{\text{get_subsystem}}) = \mathcal{O}\left(\frac{r}{\tilde{p}}\right).$$



To guarantee that we can consistently find such subsystems within the available sample, we impose the constraint:

$$\frac{r}{\bar{p}} < \binom{m}{\mu}.$$

To rearrange the subsystems, as well as to reorder the equations within each subsystem based on variable overlap, the required time complexities are $\mathcal{O}(r^2)$ and $\mathcal{O}(r\mu^2)$, respectively, both of which are negligible.

Now consider any subsystem S_j with at most v variables and μ equations. Algorithm 5 solves this subsystem by solving all μ equations and merging them iteratively. Let $\mathcal{T}_j^i$ denote the size of the table that stores the solutions after merging the first i equations of the subsystem. Let ν_i represent the number of distinct variables involved in a random selection of i equations from S_j . The expected value of ν_i can be derived from the analysis presented in Appendix B. Due to the imposed maximal variable overlap condition, with high probability the actual number of variables involved in the first i equations is at most ν_i . Therefore, we can consider the expected size of the table $\mathbb{E}[\mathcal{T}_j^i] = \mathcal{O}\left(\frac{(2B+1)^{\nu_i+i}}{M^i}\right)$.

To solve the subsystem, at the $(i+1)^{\text{th}}$ step, Algorithm 5 solves the $(i+1)^{\text{th}}$ equation using Algorithm 2 and then merges the solutions of the first i equations with the solution of the $(i+1)^{\text{th}}$ equation using Algorithm 3. This step produces the solutions for the first $(i+1)^{\text{th}}$ equations. Hence, from the

analysis of **Merge Algorithm**, the time complexity to merge these tables is given by:

$$\mathcal{T}_{\text{merge}} = \mathcal{O} \left(\max \left\{ \mathcal{T}_j^1, \mathcal{T}_j^i, \mathcal{T}_j^{i+1} \right\} \right).$$

So the time complexity to merge all the solutions of every equation is $\mathcal{O} \left(\max_i \left\{ \mathcal{T}_j^i \right\} \right)$. From the analysis of **Solve Algorithm**, the time complexity to solve a single equation is

$$\mathcal{T}_{\text{solve}} = \mathcal{O} \left(\max \left\{ (2B+1)^\kappa, \frac{(2B+1)^{k+1}}{M} \right\} \right),$$

where $\kappa = \lceil \frac{k+1}{2} \rceil$, and so the time complexity to solve all μ equations is $\mu \mathcal{T}_{\text{solve}}$. Therefore, the overall time complexity to solve a single subsystem is:

$$\mathcal{T}_{\text{solve_subsystem}} = \mathcal{O} \left(\max \left\{ \mu \mathcal{T}_{\text{solve}}, \max_{i,j} \left\{ \mathcal{T}_j^i \right\} \right\} \right)$$

and the memory complexity is:

$$\mathcal{M}_{\text{solve_subsystem}} = \mathcal{O} \left(\max_{i,j} \left\{ \mathcal{T}_j^i \right\} \right).$$

Hence, the time complexity to solve all r subsystems becomes:

$$r \mathcal{T}_{\text{solve_subsystem}}.$$

The expected size of the solutions for each subsystem is given by $\frac{(2B+1)^{v+\mu}}{M^\mu}$. Now, Algorithm 5 merges the solutions of subsystems iteratively. Let $\mathcal{T}_j$ denote the number of merged solutions after processing the first j subsystems. According to Theorem 1, the expected number of distinct variables across j randomly selected subsystems is given by

$$v_j = n \left[1 - \left(1 - \frac{v}{n} \right)^j \right].$$

Since we enforce the maximal variable overlap condition, the actual number of variables involved in the first j subsystems is, with high probability, less than or equal to v_j . Consequently, we can consider the expected number of merged solutions

$$\mathbb{E}[\mathcal{T}_j] = \mathcal{O} \left(\frac{(2B+1)^{v_j+j\mu}}{M^{j\mu}} \right).$$

At the $(j+1)^{\text{th}}$ step, Algorithm 5 merges the solutions of the first j subsystems with the $(j+1)^{\text{th}}$ subsystem. This step produces the merged solutions for the first $(j+1)$ subsystems. Hence, from the analysis of the **Merge Algorithm**, the time complexity of the $(j+1)^{\text{th}}$ step is given by:

$$\mathcal{O} \left(\max_j \left\{ \mathcal{T}_1, \mathcal{T}_j, \mathcal{T}_{j+1} \right\} \right).$$

Therefore, the time complexity to merge the solutions of all subsystems is:

$$\mathcal{T}_{\text{all_merge}} = \mathcal{O} \left(\max_j \left\{ \mathcal{T}_j \right\} \right)$$

and the memory complexity is:

$$\mathcal{M}_{\text{all_merge}} = \mathcal{O} \left(\max_j \left\{ \mathcal{T}_j \right\} \right).$$

The next step is to solve some equations after putting the values of known variables. Since for these equations, the number of unknown variables to solve is less than $\delta(< k)$, the complexities for solving these equations are negligible.

Hence, the overall time complexity to solve a bounded sLWE instance with sparsity above the threshold becomes:

$$\max\{r\mathcal{T}_{\text{get_subsystem}}, r\mathcal{T}_{\text{solve_subsystem}}, \mathcal{T}_{\text{all_merge}}\}$$

and the overall memory complexity becomes:

$$\max\{\mathcal{M}_{\text{solve_subsystem}}, \mathcal{M}_{\text{all_merge}}\}.$$

We will select the optimal values for v, μ and r to minimize the complexity, subject to the following constraints:

1. $\frac{r}{p} < \binom{m}{\mu}$
2. $(2B+1)^{v'+r\mu} \leq M^{r\mu}$
3. $n \left[1 - \left(1 - \frac{v'}{n} \right) \left(1 - \frac{k}{n} \right) \right] - v' < \delta,$

where $v' = n \left[1 - \left(1 - \frac{v}{n} \right)^r \right]$.

7 Numerical Results

To illustrate the concrete performance of our algorithms, we present numerical results for the bounded sLWE problem in the ternary case ($B = 1$). This setting represents the smallest nontrivial bound and provides the most favorable regime for the proposed combinatorial techniques.

We evaluate the concrete security of ternary sLWE by examining how the parameter sets proposed in [13] behave when translated to the ternary setting. The recommended dimensions for sLWE with target security comparable to dimension-1024 LWE are listed in Table 2 for moduli 2^{64} , 2^{32} , and 2^{16} .

Table 2: Recommended sLWE parameters from [13] targeting security comparable to dimension-1024 LWE for moduli 2^{64} , 2^{32} , and 2^{16} .

Sparsity Parameter k	Number of Samples m		
	2^{13}	2^{17}	2^{21}
20	1218	1425	1656
30	1143	1265	1395
40	1110	1195	1285
50	1090	1158	1234

As reference points, we compare our results with two well-established attacks on ternary LWE of dimension 1024:

- The MEET-LWE [17] algorithm, whose run time is at least $S^{0.25} = 3^{1024 \times 0.25} \approx 2^{406}$.
- Primal lattice reduction, for which we used the lattice estimator [1] to obtain concrete complexity exponents. The resulting estimates are given in Table 3.

We apply our analysis to the parameter sets listed in Table 2 in the ternary setting. The resulting complexity estimates are summarized in Tables 4 and 5, corresponding to the regimes below and above the sparsity threshold identified in our analysis. For clarity, we classify the outcomes as follows: entries marked in “green” correspond to instances where our attack is faster than primal lattice reduction;

Table 3: Time exponents for primal lattice reduction on ternary LWE with dimension 1024, estimated using the lattice estimator.

Modulus	Time (exponent)
2^{16}	198
2^{32}	102
2^{64}	54

entries in “yellow” indicate complexities larger than lattice reduction but smaller than MEET-LWE; and entries in “red” correspond to complexities exceeding that of MEET-LWE.

Our results show that, for modulus 2^{64} , the translated ternary sLWE parameters admit combinatorial attacks with complexity significantly below the lattice-reduction baseline. For moduli 2^{32} and 2^{16} , we observe a noticeable reduction in the effective security margin in several configurations, although some parameter choices remain outside the practical reach of our attack when targeting a 128-bit security level.

Table 4: Summary of time and memory complexity exponents for ternary sLWE instances with sparsity at or below the threshold limit.

M	k	$m = 2^{13}$			$m = 2^{17}$			$m = 2^{21}$		
		n	Time (exponents)	Memory (exponents)	n	Time (exponents)	Memory (exponents)	n	Time (exponents)	Memory (exponents)
2^{64}	20	1218	25	18	1425	25	18	1656	29	21
	30	1143	32	26	1265	32	26	1395	32	26
	40	1110	40	34	1195	40	34	1285	40	34
2^{32}	20	1218	25	18	1425	25	18	1656	29	21

Table 5: Summary of time and memory complexity exponents for ternary sLWE instances with sparsity above the threshold limit.

M	k	$m = 2^{13}$						$m = 2^{17}$						$m = 2^{21}$					
		n	v	μ	r	Time (exponents)	Memory (exponents)	n	v	μ	r	Time (exponents)	Memory (exponents)	n	v	μ	r	Time (exponents)	Memory (exponents)
2^{64}	50	1090	92	2	8	46	42	1158	92	2	8	46	42	1234	92	2	9	46	42
2^{32}	30	1143	186	8	4	57	55	1265	208	9	4	63	61	1395	209	9	4	69	66
	40	1110	289	11	4	134	132	1195	288	11	4	134	134	1285	313	12	4	144	142
	50	1090	387	13	4	230	228	1158	375	13	4	217	215	1234	373	13	4	215	213
2^{16}	20	1218	302	24	4	149	147	1425	352	28	4	172	171	1656	414	33	4	198	197
	30	1143	421	28	4	287	285	1265	448	30	4	306	304	1395	492	33	4	338	332
	40	1110	528	29	4	440	440	1195	515	30	4	417	416	1285	546	32	4	441	441
	50	1090	611	29	4	569	567	1158	600	30	5	549	547	1234	591	31	5	534	533

For larger bounds, we also evaluated the cases $B = 2, 3$, and 4 with modulus 2^{64} . The corresponding threshold limits are 27, 22, and 20, respectively. Using the lattice estimator, the estimated complexities

of primal lattice reduction for B -bounded LWE with dimension 1024 are 2^{55} , 2^{56} , and 2^{57} for $B = 2, 3$, and 4, respectively.

Applying Algorithm 4 to the parameter set $(n, m, k, M) = (1218, 2^{13}, 20, 2^{64})$, the resulting time complexities for solving the B -bounded sLWE instances are approximately 2^{33} , 2^{38} , and 2^{42} for $B = 2, 3$, and 4, respectively. These estimates indicate that, for these bounds, the translated sLWE parameters admit combinatorial attacks with substantially lower complexity than the corresponding dimension-1024 bounded LWE instances.

Overall, these results provide a quantitative picture of how the proposed parameter sets behave in the bounded setting and identify parameter regions where the complexity is most affected by our analysis.

8 Experimental verification

We present a proof-of-concept implementation to illustrate the practical performance of our proposed techniques in the ternary setting ($B = 1$), which represents the simplest nontrivial bounded case. All experiments were conducted in SageMath 10.4 on a 24-core Intel Core i9 processor operating at 4.1 GHz. The experimental setup and results are summarized below.

Experimental Verification of the Algorithm to Solve Ternary sLWE Instances with Sparsity Parameter Less Than or Equal to the Threshold Limit: To assess the performance of our algorithm, we consider ternary sLWE parameters $n = 1218$, $k = 20$, $m = 2^{13}$, and modulus $M = 2^{32}$. The corresponding threshold value is $\delta = \lfloor \log_3 M \rfloor = 20$.

Using these parameters, we generated 100 independent ternary sLWE instances and applied our algorithm to recover the corresponding secret vectors. The results reported below represent averages over all experiments.

For each instance, we first employed the greedy subsystem-construction procedure to obtain a subsystem covering all variables. The average number of equations required to achieve full coverage was 98.77, which is well below the theoretical upper bound of 127 given in Table 1. Subsequently, the equations were reordered to maximize variable overlap, after which they were solved and merged iteratively. On average, unique recovery of the secret was achieved after solving 9.22 equations, which closely matches the theoretical estimate of 9 equations derived in Appendix B.

In all 100 instances, the algorithm successfully recovered the correct ternary secret vector, thereby confirming the correctness of our analysis and the effectiveness of the proposed approach in this parameter regime.

Experimental Verification of the Algorithm to Solve Ternary sLWE Instance with Sparsity Parameter Above the Threshold Limit: To evaluate the performance of our algorithm in the regime of sparsity above the threshold, we consider ternary sLWE parameters $n = 220$, $k = 16$, $m = 2^{10}$, and modulus $M = 2^{16}$. Using these parameters, we generated 100 independent ternary sLWE instances and attempted to recover the corresponding secret vectors using our algorithm. The values reported below are averaged over all experiments.

We constructed subsystems consisting of $v = 47$ variables and $\mu = 4$ equations, with parameter $r = 4$. By Theorem 1, the theoretical expected number of variables appearing in four randomly chosen equations is 57.35 with variance 4.7. Approximating this distribution by a normal distribution with the same mean and variance, the probability of obtaining a subsystem involving at most 47 variables is approximately $2^{-16.94}$. In practice, across all experiments, the average number of trials required to identify such a subsystem was $53,531.15 \approx 2^{15.71}$.

After identifying suitable subsystems, we rearranged their equations as well as the subsystems to maximize variable overlap. Each subsystem was then solved by first enumerating solutions of individual equations and subsequently merging them iteratively using Algorithm 2 and Algorithm 3.

Across all subsystems, the average number of solutions to the first equation was 1969.10, closely matching the theoretical estimate $\frac{3^{16+1}}{2^{16}} \approx 1971$. For the first two equations, the average number of distinct variables involved was 27.5, which is below the theoretical expectation of 29, due to the deliberate rearrangement maximizing variable overlap. The average number of merged solutions after combining the first two equations was 47,763.88, smaller than the theoretical estimate $\frac{3^{29+2}}{2^{16 \cdot 2}} \approx 143,813$, demonstrating the effectiveness of the overlap-based ordering. For the first three equations, the average number of variables was 37.53, compared to the theoretical expectation of 39. The average number of merged solutions was 161,809.11, while the theoretical estimate is $\frac{3^{39+3}}{2^{16 \cdot 3}} \approx 388,734$. Again, the reduction reflects the impact of maximizing variable overlap. For the complete subsystem of four equations, the average number of merged solutions was 105,598.18, which aligns well with the theoretical estimate $\frac{3^{47+4}}{2^{16 \cdot 4}} \approx 116,751$. The slightly smaller experimental value arises because some subsystems involved fewer than 47 variables.

Next, we merged multiple subsystems. For the union of the first two subsystems, the average number of distinct variables was 81.47, compared to the theoretical value $n \left[1 - \left(1 - \frac{v}{n} \right)^2 \right] \approx 84$. The average number of merged solutions was 144,968.6, below the theoretical estimate $\frac{3^{84+8}}{2^{16 \cdot 8}} \approx 230,842$, again confirming the benefit of structured overlap. For the union of three subsystems, the average number of variables was 109.86, slightly below the theoretical expectation $n \left[1 - \left(1 - \frac{v}{n} \right)^3 \right] \approx 113$. The average number of merged solutions was 35.92, compared to the theoretical estimate $\frac{3^{113+12}}{2^{16 \cdot 12}} \approx 69$.

Finally, after merging all subsystems, the average number of remaining candidate solutions was 1.43. For each experiment, we then selected on average 13.53 additional equations such that the number of non-overlapping variables with the merged subsystem was less than the threshold of 10. Solving these additional equations iteratively enabled us to successfully recover the ternary secret vector in every case.

9 Conclusion

We have presented a systematic cryptanalytic study of bounded sparse Learning With Errors. Our analysis identifies a sparsity threshold that separates two distinct algorithmic regimes: a regime in which individual noisy equations can be handled independently, and a regime in which structured subsystem extraction and coordinated merging become necessary. This threshold perspective provides a useful framework for understanding how sparsity influences the complexity of solving bounded sLWE instances across different bounds.

In addition to the algorithmic analysis, we examined how the parameter sets proposed for standard sLWE behave when translated to the bounded setting. Our results indicate that sparsity has a significant impact on the concrete complexity of these instances and must be taken into account when evaluating security in bounded-secret regimes. The ternary case ($B = 1$) provides a representative example where the resulting complexities can differ substantially.

More broadly, this work shows that the hardness of sparse LWE-type constructions depends sensitively on restrictions placed on the secret and error distributions. We expect that the threshold-based viewpoint developed in this work will be useful for further investigations of sparse lattice-based assumptions and for guiding parameter selection in efficiency-oriented designs.

Acknowledgment. The authors would like to thank Steven Galbraith for his valuable comments on the initial version of this paper.

Author Contributions. A.K. and S.S. wrote the main manuscript. S.S. and W.M. supervised A.K. All authors reviewed the manuscript.

References

1. Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of Learning with Errors. volume 9, pages 169–203. 2015.
2. H. Baan, S. Bhattacharya, S. Fluhrer, O. Garcia-Morchon, T. Laarhoven, R. Rietman, M.-J. O. Saarinen, L. Tolhuizen, and Z. Zhang. Round5: Compact and fast post-quantum public-key encryption. In *International Conference on Post-Quantum Cryptography*, page 83–102. Springer, 2019.
3. Shi Bai, Steven D. Galbraith, Liangze Li, and Daniel Sheffield. Improved Exponential-time Algorithms for Inhomogeneous-SIS. *IACR Cryptology ePrint Archive*, 2014:593, 2014.
4. D. J. Bernstein, C. Chuengsatiansup, T. Lange, and C. van Vredendaal. NTRU prime: Reducing attack surface at low cost. In C. Adams and J. Camenisch, editors, *SAC 2017*, volume 10719 of *LNCS*, page 235–260. Springer, Heidelberg, 2017.
5. J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé. Crystals-kyber: a cca-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, page 353–367. IEEE, 2018.
6. J.-P. Bossuat, C. Mouchet, J. Troncoso-Pastoriza, and J.-P. Hubaux. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, page 587–617. Springer, 2021.
7. C. Chen, O. Danba, J. Hoffstein, A. Hülsing, J. Rijneveld, J. M. Schanck, T. Saito, P. Schwabe, W. Whyte, K. Xagawa, T. Yamakawa, and Z. Zhang. PQC round-3 candidate: NTRU. Technical report, 2020. Accessed: 2022-05-20.
8. L. Ducas, A. Durmus, T. Lepoint, and V. Lyubashevsky. Lattice signatures and bimodal gaussians. In *Annual Cryptology Conference*, page 40–56. Springer, 2013.
9. C. Gentry. Fully homomorphic encryption using ideal lattices. In M. Mitzenmacher, editor, *41st ACM STOC*, page 169–178. ACM Press, 2009.
10. T. Güneysu, V. Lyubashevsky, and T. Pöppelmann. Practical lattice-based cryptography: A signature scheme for embedded systems. In *International Workshop on Cryptographic Hardware and Embedded Systems*, page 530–547. Springer, 2012.
11. J. Hoffstein, J. Pipher, and J. H. Silverman. NTRU: A ring-based public key cryptosystem. In J. P. Buhler, editor, *Algorithmic Number Theory*, page 267–288. Springer Berlin Heidelberg, Berlin, Heidelberg, 1998.
12. N. Howgrave-Graham. A hybrid lattice-reduction and meet-in-the-middle attack against NTRU. In A. Menezes, editor, *Advances in Cryptology - CRYPTO 2007*, page 150–169. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007.
13. A. Jain, H. Lin, and S. Saha. A systematic study of sparse LWE. In L. Reyzin and D. Stebila, editors, *Advances in Cryptology - CRYPTO 2024*, volume 14922 of *Lecture Notes in Computer Science*, page to appear. Springer, Cham, 2024.
14. J.-W. Lee, E. Lee, Y. Lee, Y.-S. Kim, and J.-S. No. High-precision bootstrapping of RNS-CKKS homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, page 618–647. Springer, 2021.
15. Y. Lee, J.-W. Lee, Y.-S. Kim, Y. Kim, J.-S. No, and H. Kang. High-precision bootstrapping for approximate homomorphic encryption by error variance minimization. In *Advances in Cryptology-EUROCRYPT 2022: 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30–June 3, 2022, Proceedings, Part I*, page 551–580. Springer, 2022.
16. V. Lyubashevsky. Lattice signatures without trapdoors. In D. Pointcheval and T. Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, page 738–755. Springer, Heidelberg, 2012.
17. A. May. How to meet ternary LWE keys. In T. Malkin and C. Peikert, editors, *Advances in Cryptology - CRYPTO 2021*, page 701–731. Springer International Publishing, Cham, 2021.
18. C. Peikert. Public-key cryptosystems from the worst-case shortest vector problem: extended abstract. In M. Mitzenmacher, editor, *41st ACM STOC*, page 333–342. ACM Press, 2009.
19. O. Regev. On lattices, learning with errors, random linear codes, and cryptography. In *STOC*, page 84–93, 2005.
20. Richard Schroeppel and Adi Shamir. A $T = O(2^{n/2})$, $S = O(2^{n/4})$ Algorithm for Certain NP-Complete Problems. *SIAM J. Comput.*, 10:456–464, 1981.
21. D. Stehlé, R. Steinfeld, K. Tanaka, and K. Xagawa. Efficient public key encryption based on ideal lattices. In M. Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, page 617–635. Springer, Heidelberg, 2009.

Appendix-A

Theorem 2 Let $A_0, A_1, \dots, A_{\mu-1}$ be random subsets of the set $[n]$ such that $|A_i| = k_i$ for each $i = 0, 1, \dots, \mu-1$. Define the random variable X as the size of the union $A_0 \cup A_1 \cup \dots \cup A_{\mu-1}$. Then, the expected value of X is

given by

$$\mathbb{E}(X) = n \left[1 - \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right) \right],$$

and the variance of X can be expressed as:

$$\mathbb{V}(X) = n \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right) - n^2 \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right)^2 + n(n-1) \prod_{j=0}^{\mu-1} \left(1 - \frac{2k_j}{n} + \frac{k_j(k_j-1)}{n(n-1)} \right).$$

Proof Let $A = A_0 \cup A_1 \cup \dots \cup A_{\mu-1}$. To analyze the union, we introduce indicator random variables X_{ij} defined as follows:

$$X_{ij} = \begin{cases} 1 & \text{if } i \in A_j, \\ 0 & \text{otherwise} \end{cases}$$

for each $i = 0, 1, 2, \dots, n-1$ and $j = 0, 1, \dots, \mu-1$. It follows that for fixed i , the variables X_{ij} are independent, and we can compute their expected values as $\mathbb{E}(X_{ij}) = \frac{k_j}{n}$. Additionally, we define another indicator variable X_i for each i :

$$X_i = \begin{cases} 1 & \text{if } i \in A, \\ 0 & \text{otherwise} \end{cases}$$

Also, we have the following relations:

$$X_i = 1 - \prod_{j=0}^{\mu-1} (1 - X_{ij}) \text{ and } X = \sum_{i=0}^{n-1} X_i.$$

To find $\mathbb{E}(X)$, we calculate:

$$\mathbb{E}(X) = \sum_{i=0}^{n-1} \mathbb{E}(X_i) = n - \sum_{i=0}^{n-1} \mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij}) \right).$$

Applying the independence of the X_{ij} for a fixed i yields:

$$\mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij}) \right) = \prod_{j=0}^{\mu-1} \mathbb{E}(1 - X_{ij}) = \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right).$$

Thus, we have the expected value of X :

$$\mathbb{E}(X) = n - \sum_{i=0}^{n-1} \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right) = n \left[1 - \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right) \right].$$

Next, we will compute the variance $\mathbb{V}(X)$:

$$\begin{aligned} \mathbb{V}(X) &= \mathbb{V} \left(\sum_{i=0}^{n-1} X_i \right) \\ &= \mathbb{V} \left(n - \sum_{i=0}^{n-1} \prod_{j=0}^{\mu-1} (1 - X_{ij}) \right) \\ &= \mathbb{V} \left(\sum_{i=0}^{n-1} \prod_{j=0}^{\mu-1} (1 - X_{ij}) \right) \\ &= \sum_{i=0}^{n-1} \mathbb{V} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij}) \right) + 2 \sum_{i=0}^{n-1} \sum_{r=i+1}^{n-1} \text{COV} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij}), \prod_{j=0}^{\mu-1} (1 - X_{rj}) \right) \\ &= \sum_{i=0}^{n-1} \left[\mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij})^2 \right) - \left\{ \mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij}) \right) \right\}^2 \right] \\ &\quad + 2 \sum_{i=0}^{n-1} \sum_{r=i+1}^{n-1} \left[\mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij})(1 - X_{rj}) \right) - \mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij}) \right) \mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{rj}) \right) \right]. \end{aligned}$$

From the independence of X_{ij} for a fixed i , we have

$$\begin{aligned} \mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij})^2 \right) - \left\{ \mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij}) \right) \right\}^2 &= \prod_{j=0}^{\mu-1} \mathbb{E} ((1 - X_{ij})^2) - \prod_{j=0}^{\mu-1} \{\mathbb{E} (1 - X_{ij})\}^2 \\ &= \prod_{j=0}^{\mu-1} \mathbb{E} (1 - X_{ij}) - \prod_{j=0}^{\mu-1} \{\mathbb{E} (1 - X_{ij})\}^2 \\ &= \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right) - \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right)^2 \end{aligned}$$

Similarly, we can get

$$\mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij}) \right) \mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{rj}) \right) = \prod_{j=0}^{\mu-1} \mathbb{E} (1 - X_{ij}) \prod_{j=0}^{\mu-1} \mathbb{E} (1 - X_{rj}) = \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right)^2$$

Also,

$$\begin{aligned} \mathbb{E} \left(\prod_{j=0}^{\mu-1} (1 - X_{ij})(1 - X_{rj}) \right) &= \prod_{j=0}^{\mu-1} \mathbb{E} ((1 - X_{ij})(1 - X_{rj})) \\ &= \prod_{j=0}^{\mu-1} \mathbb{E} (1 - X_{ij} - X_{rj} + X_{ij}X_{rj}) \\ &= \prod_{j=0}^{\mu-1} (1 - \mathbb{E}(X_{ij}) - \mathbb{E}(X_{rj}) + \mathbb{E}(X_{ij}X_{rj})) \\ &= \prod_{j=0}^{\mu-1} \left(1 - \frac{2k_j}{n} + \frac{k_j(k_j - 1)}{n(n - 1)} \right) \end{aligned}$$

After substituting all these values into our variance expression, we obtain:

$$\mathbb{V}(X) = n \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right) - n^2 \prod_{j=0}^{\mu-1} \left(1 - \frac{k_j}{n} \right)^2 + n(n - 1) \prod_{j=0}^{\mu-1} \left(1 - \frac{2k_j}{n} + \frac{k_j(k_j - 1)}{n(n - 1)} \right)$$

This completes the proof of the theorem, establishing both the expected value and variance of the random variable X .

Appendix-B

Let us consider a subsystem with exactly v variables and μ equations. We aim to determine the expected number of variables present in γ equations from this subsystem. We can translate our problem into the following:

We are considering μ subsets $A_0, A_1, \dots, A_{\mu-1}$ of the set $[n]$, where each subset consists of k elements. The task is to find the expected size of the union $\bigcup_{i=0}^{\gamma-1} A_i$, given that the total number of distinct elements in the union of all μ subsets, $\bigcup_{i=0}^{\mu-1} A_i$, is equal to v . This can be expressed mathematically as

$$\mathbb{E} \left(\left| \bigcup_{i=0}^{\gamma-1} A_i \right| \mid \left| \bigcup_{i=0}^{\mu-1} A_i \right| = v \right).$$

To compute this expectation, we need to evaluate the conditional probability

$$\mathbb{P} \left(\left| \bigcup_{i=0}^{\gamma-1} A_i \right| = \ell \mid \left| \bigcup_{i=0}^{\mu-1} A_i \right| = v \right)$$

for all possible sizes ℓ . According to Bayes' theorem, we can express this conditional probability as

$$\mathbb{P}\left(\left|\bigcup_{i=0}^{\gamma-1} A_i\right| = \ell \mid \left|\bigcup_{i=0}^{\mu-1} A_i\right| = v\right) = \frac{\mathbb{P}\left(\left|\bigcup_{i=0}^{\mu-1} A_i\right| = v \mid \left|\bigcup_{i=0}^{\gamma-1} A_i\right| = \ell\right) \cdot \mathbb{P}\left(\left|\bigcup_{i=0}^{\gamma-1} A_i\right| = \ell\right)}{\sum_j \mathbb{P}\left(\left|\bigcup_{i=0}^{\mu-1} A_i\right| = v \mid \left|\bigcup_{i=0}^{\gamma-1} A_i\right| = j\right) \cdot \mathbb{P}\left(\left|\bigcup_{i=0}^{\gamma-1} A_i\right| = j\right)}.$$

To find $\mathbb{P}\left(\left|\bigcup_{i=0}^{\gamma-1} A_i\right| = j\right)$, we need to consider how the elements are drawn from a total of n distinct items when selecting k items for each of the γ subsets. The probability of obtaining exactly j distinct outcomes can be modeled using the Hypergeometric Distribution, which is appropriate for situations where we are sampling without replacement. This distribution provides the necessary framework to calculate the likelihood of achieving various distinct counts after γ draws.

When $\gamma = 1$, the calculation becomes simpler because there is only one subset to consider. In this case,

$$\mathbb{P}\left(\left|\bigcup_{i=0}^{\gamma-1} A_i\right| = j\right) = \mathbb{P}(|A_0| = j) = \begin{cases} 1, & \text{if } j = k, \\ 0, & \text{otherwise.} \end{cases}$$

To efficiently track the probabilities of different union sizes after γ draws, we introduce an array denoted as Λ_γ , which has a size of n such that $\Lambda_\gamma[j] = \mathbb{P}\left(\left|\bigcup_{i=0}^{\gamma-1} A_i\right| = j\right)$ for each j from 1 to n . This array serves as a compact representation of the probabilities associated with each possible size of the union $\bigcup_{i=0}^{\gamma-1} A_i$.

We begin by initializing the array Λ_γ to represent the state after a single draw. Since the first subset contains exactly k elements, we have k initial elements and thus we set $\Lambda_\gamma[k] = 1$, indicating that after one draw, the size of the union is exactly k . All other values in the array are initially set to zero.

The process then iterates for each of the additional $\gamma - 1$ draws. Each iteration updates Λ_γ to reflect the new possible sizes of the union after adding another subset. Let us assume that, after the i^{th} iteration, the size of $A_0 \cup A_1 \cup \dots \cup A_i$ is j . Then we calculate the probability of reaching all possible new sizes j_1 after adding the next subset A_{i+1} . The probability of transitioning from a union of size j to size j_1 is computed using Hypergeometric Distribution and update the probabilities stored in Λ_γ . This process repeats for each subsequent draw, updating the array Λ_γ to reflect the distribution of possible union sizes after each subset is added. When all γ draws are completed, the final array Λ_γ contains the probabilities for all possible sizes of the union $\bigcup_{i=0}^{\gamma-1} A_i$.

Once the array Λ_γ has been computed for the first γ draws, the next step involves calculating the probability distribution for the size of $\bigcup_{i=0}^{\mu-1} A_i$, conditioned on the size of the union $\bigcup_{i=0}^{\gamma-1} A_i$. This is done by building a matrix $\mathcal{M}$, where each row represents the possible sizes of the union $\bigcup_{i=0}^{\gamma-1} A_i$, and each column represents the possible sizes of the union $\bigcup_{i=0}^{\mu-1} A_i$.

For each possible size j of $\bigcup_{i=0}^{\gamma-1} A_i$, we compute the distribution of union sizes $\bigcup_{i=0}^{\mu-1} A_i$ by invoking the rest $\mu - \gamma$ draws with an initial size of j . This creates a probability distribution for how the union can grow from size j to different sizes after the remaining $\mu - \gamma$ subsets are added. The result is stored in the j^{th} row of the matrix $\mathcal{M}$.

Once the matrix $\mathcal{M}$ is constructed, we focus on the v^{th} column, which contains the probabilities of obtaining $\left|\bigcup_{i=0}^{\mu-1} A_i\right| = v$, for each possible initial size of $\bigcup_{i=0}^{\gamma-1} A_i$, that is $\mathcal{M}[j][v] = \mathbb{P}\left(\left|\bigcup_{i=0}^{\mu-1} A_i\right| = v \mid \left|\bigcup_{i=0}^{\gamma-1} A_i\right| = j\right)$ for all $j = 1, 2, \dots, n$. Utilizing Bayes' theorem, we can now compute $\mathbb{P}\left(\left|\bigcup_{i=0}^{\gamma-1} A_i\right| = \ell \mid \left|\bigcup_{i=0}^{\mu-1} A_i\right| = v\right)$ for all possible size ℓ and consequently we can compute the expected size of $\bigcup_{i=0}^{\gamma-1} A_i$ when it is given that $\left|\bigcup_{i=0}^{\mu-1} A_i\right| = v$.

Appendix-C

In the following, we propose an alternative method for estimating the expected number of samples μ needed for S to recover the full support of the secret vector through the greedy selection process. Let us denote the set of variables present in the i^{th} sample by A_i . At any point during the greedy selection process, let V represent the set of currently covered variables, and let I denote the indices of the samples that have been selected so far. For each A_i (i.e., $i \in [m] \setminus I$) corresponding to the remaining samples, define the random variable

$$X_i := |V \cap A_i|.$$

Consider $X = \min_{i \in [m] \setminus I} X_i$, which represents the *minimum overlap* of any candidate subset with the current coverage V .

Let $v = |V|$. We begin by computing the distribution function of X , denoted $F_X(t)$:

$$F_X(t) = \mathbb{P}(X \leq t) = 1 - \mathbb{P}(X > t) = 1 - \prod_{i \in [m] \setminus I} \mathbb{P}(X_i > t).$$

For each X_i , the probability that $|A_i \cap V| = j$ is given by the hypergeometric distribution:

$$\mathbb{P}(X_i = j) = \frac{\binom{v}{j} \binom{n-v}{k-j}}{\binom{n}{k}}.$$

Thus,

$$\mathbb{P}(X_i > t) = 1 - \sum_{j=0}^t \mathbb{P}(X_i = j) = 1 - \sum_{j=0}^t \frac{\binom{v}{j} \binom{n-v}{k-j}}{\binom{n}{k}}.$$

Combining this,

$$F_X(t) = 1 - \left[1 - \sum_{j=0}^t \frac{\binom{v}{j} \binom{n-v}{k-j}}{\binom{n}{k}} \right]^{m-|I|}.$$

Then the probability mass function (PMF) is:

$$\mathbb{P}(X = t) = \begin{cases} F_X(0), & \text{if } t = 0, \\ F_X(t) - F_X(t-1), & \text{otherwise.} \end{cases}$$

Therefore, the expected value of the minimal overlap is:

$$\mathbb{E}[X] = \sum_{t=0}^k t \cdot \mathbb{P}(X = t).$$

During each iteration of the greedy process, we select a new sample A_j such that its overlap with V is minimal. The number of newly covered elements is $k - X$, and thus the expected increase in coverage is approximately $k - \mathbb{E}[X]$, given the current size of V , i.e., $v = |V|$.

Using this analysis, the total number of steps required (including the initial step) to achieve full coverage (i.e., $|V| = n$) can be approximated, giving an estimate for the expected value of μ . Depending on parameters, this method can provide closer estimates for μ than the analytical upper bound.